Computer Organization and Architectur

(Code: CS256)

Lecture-16: Floating Number

Course Faculty

Dr. B. R. Bhowmik

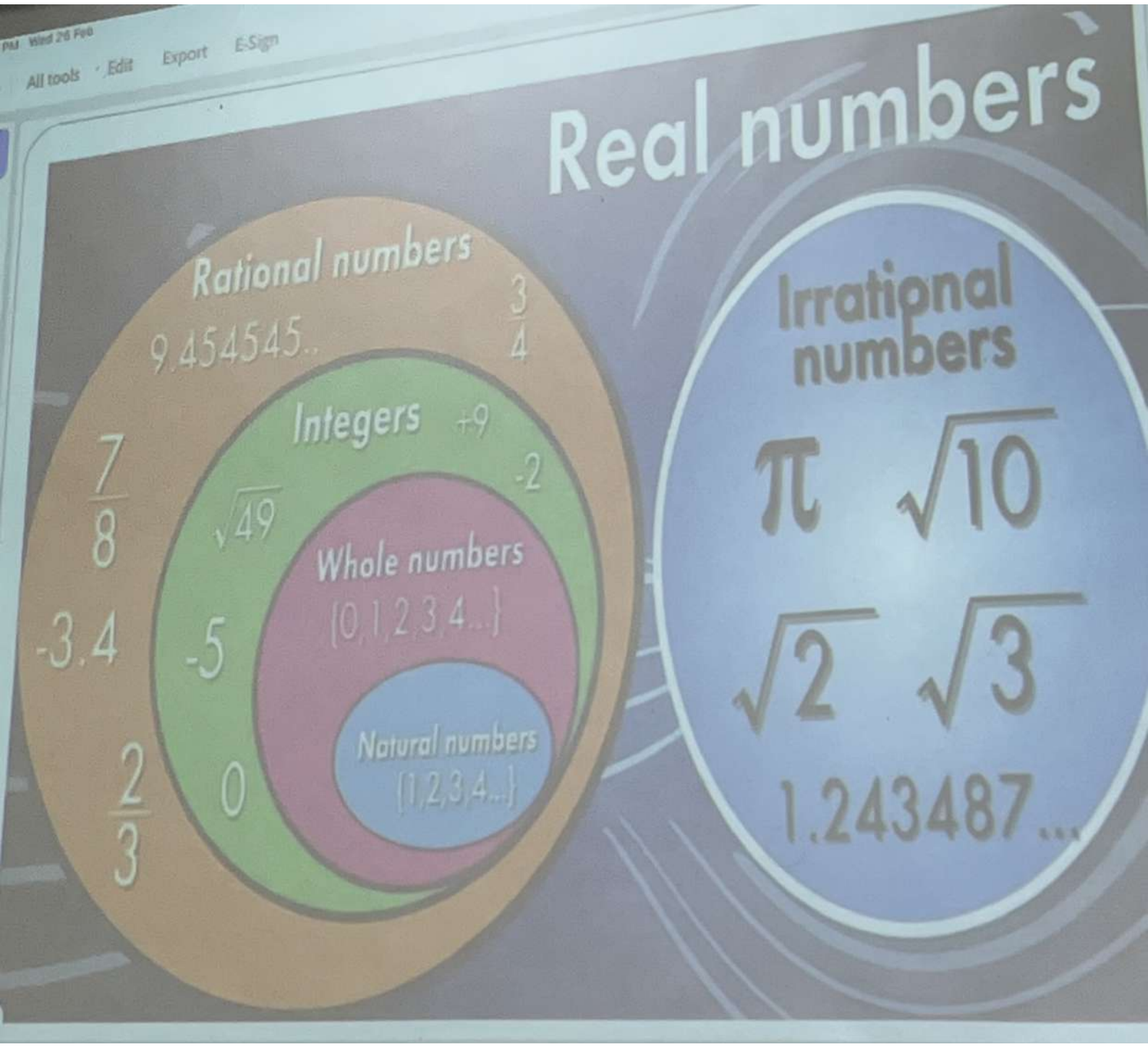
Assistant Professor

Dept. of CSE

Website: https://sites.google.com/view/biswajit-bhowmik
https://sites.google.com

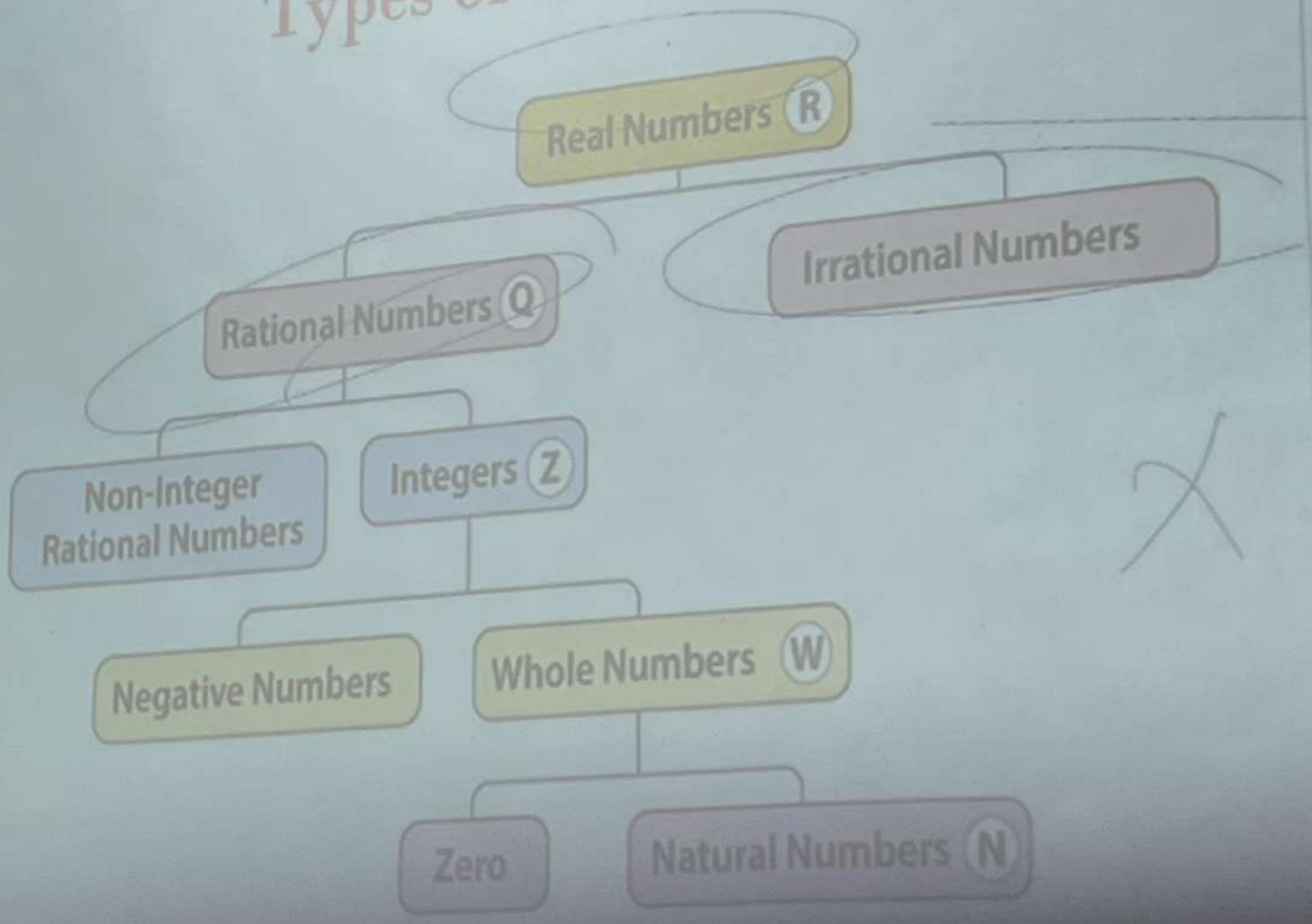
PM Wed 26 Feb
All tools · Edit Export E-Sign

Real numbers



Wed 26 Feb
All tools · Edit Export E-Sign

Types of Real Numbers



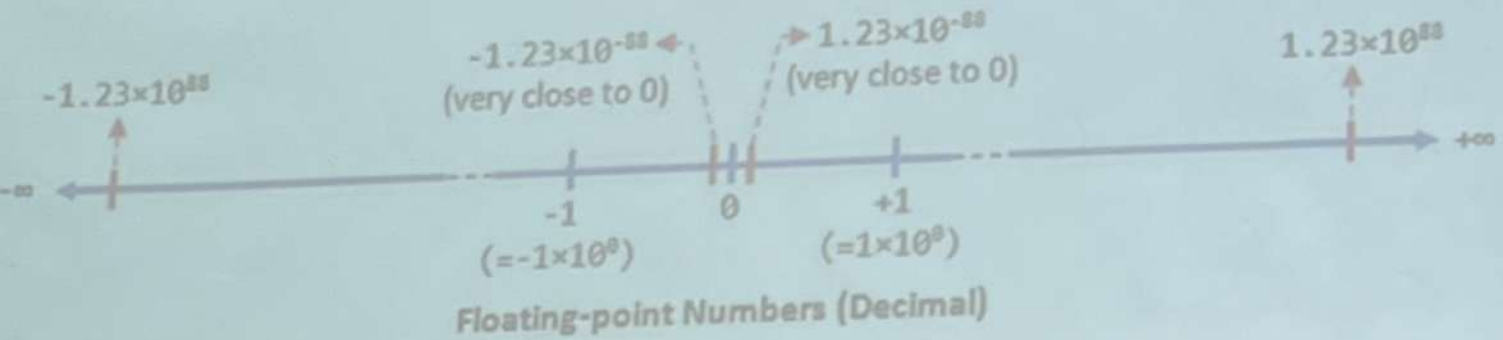
1:08 PM Wed 26 Feb
All tools · Edit Export E-Sign

Real or Fraction Number

- *Real number is a quantity that can be expressed as an infinite decimal expansion.*
- Real numbers are used in measurements of continuously varying quantities such as size and time.
- In contrast, the natural numbers 1, 2, 3, …, arising from counting.
- *A real also known as fraction number, is a whole number with a decimal point.*
- For example, 5.5, 0.25, and -103.342 are all real numbers, while 91, and 0 are not.
- *Real numbers can be defined as the union of both rational and irrational numbers.*
- They can be both positive or negative.
- They are denoted by the symbol "R".

Real Number Representations

- A real number is typically expressed in the scientific notation.
- The notation has two parts
  - A fraction (F), and
  - An exponent (E) of a certain radix (r)
- Scientific notational form: $F \times r^E$.
- Example
  - Decimal numbers use radix of 10 (F×10^E);
  - Binary numbers use radix of 2 (F×2^E).



Floating-point Numbers (Decimal)

PM Wed 26 Feb
All tools · Edit Export E-Sign

Real Number Classifications by Representation

- There are two major approaches to store real numbers (i.e., numbers with fractional component) in modern computing.
- A fraction number can be seen in
  - Fixed Point Representation
  - Floating Point Representation
- In fixed point notation, there are a fixed number of digits after the decimal point.
  - Example: 25.80, 25.00, 27.89, etc.
- Floating point number allows for a varying number of digits after the decimal point.
  - Example: 25.80, 25.001, 27.8966, etc.

Export E-Sign

Fixed Point Representation

- This representation has fixed number of bits for integer part (I) and for fractional part (F).
- For example, if given fixed-point representation is IIII.FFFF, then you can store minimum value is 0000.0001 and maximum value is 9999.9999.
- There are three parts of a fixed-point number representation:
  - sign field,
  - integer field, and
  - fractional field.

Sign Bit	Integer Part	Fractional Part

Signed Fixed point representation

Integer Part	Fractional Part

Unsigned Fixed point representation

Fixed Point Advantage and Disadvantage

- The advantage of using a fixed-point representation is performance.
- Main disadvantage is relatively limited range of values that they can represent.
- *So, it is usually inadequate for numerical analysis as it does not allow enough numbers and accuracy.*
- *A number whose representation exceeds 32 bits would have to be stored inexactly.*
- *It is important to note that real numbers suffer from loss of precision when represented with a fixed number of bits* (e.g., 32-bit or 64-bit).

1:24 PM Wed 26 Feb
All tools · Edit Export E-Sign

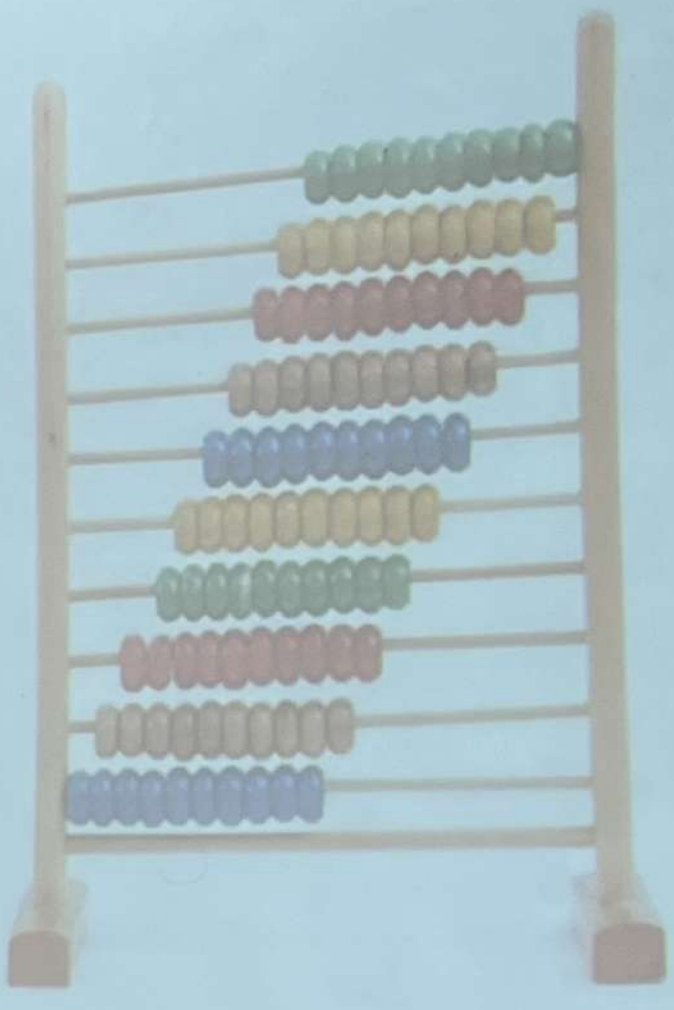
Fixed Point Advantage and Disadvantage

- This is because there are infinite number of real numbers (even within a small range of says 0.0 to 0.1).
- On the other hand, a n-bit binary pattern can represent a finite 2^n distinct numbers.
- Hence, not all the real numbers can be represented.
- The nearest approximation will be used instead, resulted in *loss of accuracy*.
- Also, floating number arithmetic is very much less efficient than integer arithmetic.
- *It could be speed up with a so-called dedicated floating-point co-processor.*
- Hence, use integers if application does not require floating-point numbers.

All tools · Edit Export

Floating Point Number

- Real number are also known as Floating point numbers due to varying position of the decimal point.
- We can move the radix point either left or right.
- Floating point numbers get their name from the way the decimal point can "float" to any position necessary.
- Due to this, floating point numbers are often referred to as floats.



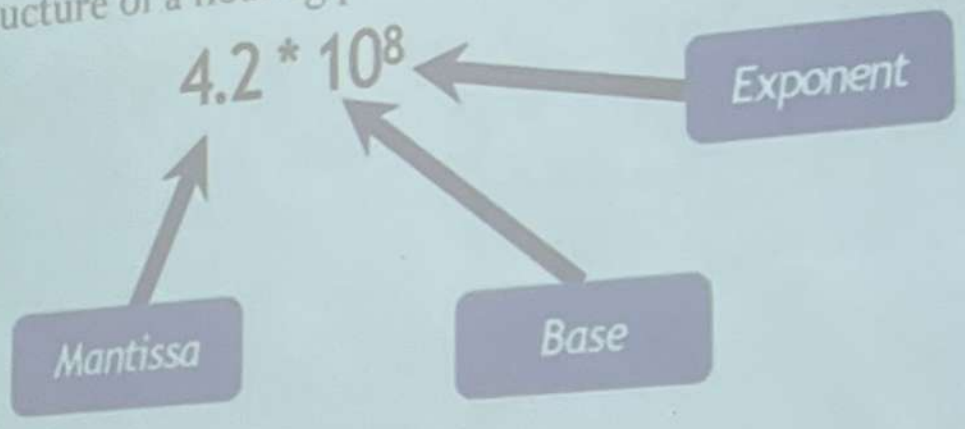
Edit Export E-Sign

Why Floating Point?

- *To an engineer* building a highway, it does not matter whether it's 10 meters or 10.0001 meters wide - their measurements are probably not that accurate in the first place.
- *To someone designing* a microchip, 0.0001 meters (a tenth of a millimeter) is a huge difference - But they'll never have to deal with a distance larger than 0.1 meters.
- To satisfy the engineer and the chip designer, a number format has to provide accuracy for numbers at very different magnitudes.
- However, only relative accuracy is needed.
- Basically, having a fixed number of integer and fractional digits is not useful - and the solution is a format with a floating point.

Structure of Floating Point Number

- The structure of a floating point(real) number is as follows:



- Only the mantissa and the exponent are stored.
- The base is implied (known already)
- As it is not stored this will save memory capacity

ools Edit Export E-Sign

Floating-Point Decimal Number

$$
\begin{aligned}
123456.\times 10^{-1} &= 12345.6\times 10^{0} \\
&= 1234.56\times 10^{1} \\
&= 123.456\times 10^{2} \\
&= 12.3456\times 10^{3} \\
&= 1.23456\times 10^{4} \text{(normalised)} \\
&\approx 0.12345\times 10^{5} \\
&\approx 0.01234\times 10^{6}
\end{aligned}
$$

All tools Edit Export E-Sign

How Floating-point Numbers Work

- Computers use a form of scientific notation for floating-point representation.
- Numbers written in scientific notation have **three** components.
- A ***significand*** or **mantissa** that contains the number's digits.
- Negative significands represent negative numbers.
- An **exponent** that says where the decimal (or binary) point is placed relative to the beginning of the significand.
- Negative exponents represent numbers that are very small (i.e. close to zero).
- Third part includes **sign** bit.

Sign Mantissa Exponent

$$+ \; 1.25 \times 10^{-1}$$

Wed 26 Feb
All tools Edit Export E-Sign

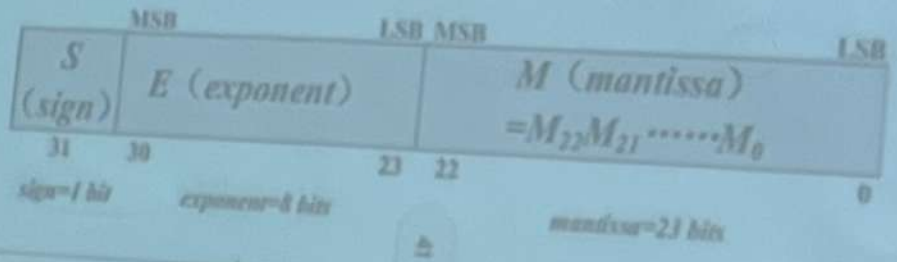
IEEE Standard for FP Numbers

- Modern computers, including Intel-based PC's, Macs, and most Unix platforms adopt IEEE 754 standard for representing floating-point numbers.
- There is a IEEE technical standard that defines the structure of a floating point number.
- IEEE Standard for Floating-Point Arithmetic (IEEE 754) in 1985.
- The standard addressed many problems found in the diverse floating point implementations that made them difficult to use reliably and reduced their portability.

All tools Edit Export E-Sign

IEEE 754 for FP Numbers: Classification

- According to IEEE 754 standard, the floating-point number is represented in **four** ways: 16, 32, 64 and 128 bit.
- Sometimes referred to as Half, Single, Double and Quadruple precision.
- *Half Precision (16 bit):* 1 sign bit, 5 bit exponent, and 10 bit mantissa
- *Single Precision (32 bit):* 1 sign bit, 8 bit exponent, and 23 bit mantissa
- *Double Precision (64 bit):* 1 sign bit, 11 bit exponent, and 52 bit mantissa
- *Quadruple Precision (128 bit):* 1 sign bit, 15 bit exponent, and 112 bit mantissa



Four IEEE 754 Formats

Exponent +bias

Half precision (binary16)

Single precision (binary32)

fraction (23 bits)

- 0.15625

Double precision (binary64)

Quadruple precision (binary128)

IEEE Standard for FP Numbers

- Nearly all hardware and programming languages use floating-point numbers in the same binary formats, which are defined in the IEEE 754 standard.
- Developed in response to divergence of representations
  - Portability issues for scientific code
- Now almost universally adopted

Format	Total bits	Significand bits	Exponent bits	Smallest number	Largest number
Single precision	32	23 + 1 sign	8	ca. $1.2 \cdot 10^{-38}$	ca. $3.4 \cdot 10^{38}$
Double precision	64	52 + 1 sign	11	ca. $2.2 \cdot 10^{-308}$	ca. $1.8 \cdot 10^{308}$

All tools · Edit Export

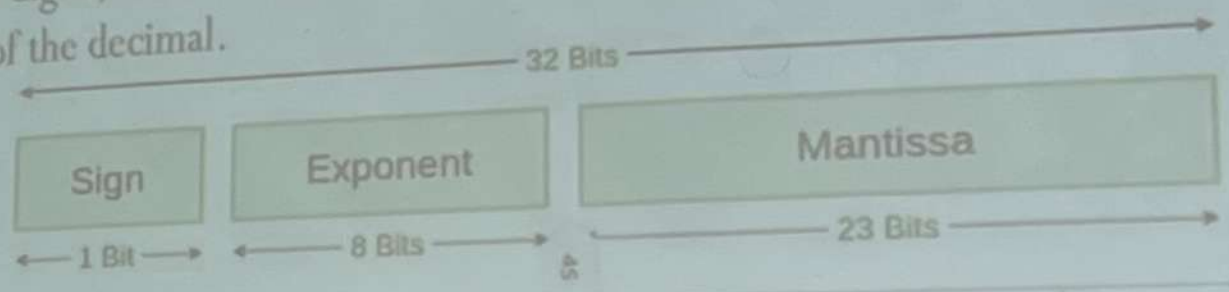
IEEE 754: FP Standard

- Standard for floating point storage
- 32 and 64 bit standards
- 8 and 11 bit exponent respectively
- Extended formats (both mantissa and exponent) for intermediate results
- Representation: sign, exponent, faction
  - 0: 0, 0, 0
  - -0: 1, 0, 0
  - Plus infinity: 0, all 1s, 0
  - Minus infinity: 1, all 1s, 0
  - NaN; 0 or 1, all 1s, =! 0

Edit Export E-Sign

IEEE 754 Single Precision

- In 32-bit single-precision floating-point representation:
- The most significant bit is the sign bit (S), with 0 for positive numbers and 1 for negative numbers.
- The following 8 bits represent exponent (E).
- The remaining 23 bits represents fraction (F).
- ***The Biased exponent*** –The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.
- ***The Normalized Mantissa*** –The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits. Here we have only 2 digits, i.e. O and 1. So a normalised mantissa is one with only one 1 to the left of the decimal.



All tools · Edit Export E-Sign

IEEE 754 Single Precision

- Let the sign bit (31) be s, the exponent (30-23) be e and the mantissa (significand or fraction) (22-0) be m.
- The valid range of the exponents is 1 to 254 (if e is treated as an unsigned number).
- The actual exponent is biased by 127 to get e i.e. the actual value of the exponent is $e - 127$.
- This gives the range: $2^{1-127} = 2^{-126}$ to $2^{254-127} = 2^{127}$.

tools · Edit Export

IEEE 754 Single Precision Normalization

- The normalized significand is $1.m$ (binary dot). The binary point is before bit-22 and the 1 (one) is not present explicitly.
- The sign bit $s = 1$ for a $-$ve number is zero (0) for a $+$ve number.
- The value of a normalized number is

$$(-1)^s \times 1.m \times 2^{e-127}$$

An Example

Consider the following 32-bit pattern

1 1011 0110 011 0000 0000 0000 0000 0000

The value is

127

$$(-1)^{1} \times 2^{10110110-01111111} \times 1.011$$

$$= -1.375 \times 2^{55}$$

$$= -49539595901075456.0$$

$$= -4.9539595901075456 \times 10^{16}$$

An Example

Consider the decimal number: + 105.625. The equivalent binary representation is

$$+1101001.101$$

$$= +1.101001101 \times 2^{6}$$

$$= +1.101001101 \times 2^{133-127}$$

$$= +1.101001101 \times 2^{10000101-01111111}$$

In IEEE 754 format:

0 1000 0101 101 0011 0100 0000 0000 0000

A 32-bit Floating Point Format

0.11010001	2^{10100}	= 0 10010011 10100010000000000000000
-0.11010001	2^{10100}	= 1 10010011 10100010000000000000000
0.11010001	2^{-10100}	= 0 01101011 10100010000000000000000
-0.11010001	2^{-10100}	= 1 01101011 10100010000000000000000

(b) Examples

Example

- Express 85.125 in single precision format

  85 = 1010101

  0.125 = 001

  85.125 = 1010101.001

  =1.010101001 x 2^6

  sign = 0

- biased exponent 127+6=133
- 133 = 10000101
- Normalized mantissa = 010101001
- we will add 0's to complete the 23 bits
- The IEEE 754 Single precision is:

  0 10000101 01010100100000000000000

- This can be written in hexadecimal form 42 A4000

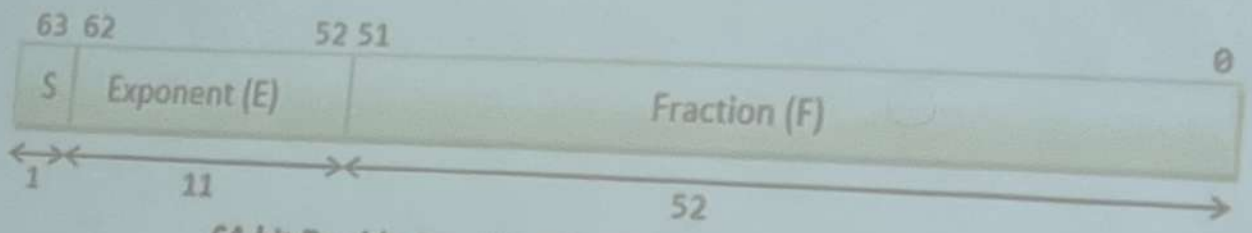
Special Values

- For single-precision floating-point, exponents in the range of -126 to + 127 are biased by adding 127 to get a value in the range 1 to 254 (0 and 255 have special meanings).
- An exponent of 255 indicates a special value.
- If the significand is zero, the value is infinity.
- If the significand is nonzero, the value is Na. N, "not a number," often used to flag an error condition.

All tools Edit Export E-Sign

IEEE 754 Double Precision

- Double precision binary floating-point is a commonly used format on PCs, due to its wider range over single precision floating point, even if at a performance and bandwidth cost.
- As with single precision floating point format, it lacks precision on integer numbers when compared with an integer format of the same size.
- It is commonly known simply as double. The IEEE 754 standard defines a double as:
- Sign bit: 1 bit ; Exponent width: 11 bits ; Significand precision: 52 bits



64-bit Double-Precision Floating-point Numl